

《矮人要塞》（Dwarf Fortress）

世界生成机制

矮人要塞的世界生成极为复杂，整个地图由若干“区域”格子构成，每个区域由16×16个本地区块组成，每个本地区块又由48×48个游戏格子构成¹。生成开始时，游戏首先根据玩家设定的参数（世界大小、极地位置等）分配内存，并在区域格子上以网格方式播撒基本地形参数（海拔、降雨、温度、排水、火山活动、野性度等），然后采用分形算法插值填充各区域地形²。极地效应对温度场产生梯度变化（如南极温暖模式或北极冰冷模式）。完成第一轮播撒后，系统修正部分高度以满足参数要求，若不符合则拒绝此次世界并重新生成²。

接下来生成植被和生物群落：根据地形参数（温度、降雨、海拔等）决定各地的植被类型，并检测生物群落分布是否满足参数要求³。随后进入侵蚀阶段：系统在山地边缘随机选点作为“降雨源”，模拟溪流向海洋流淌并切割地形，同时填充湖泊、平滑极端高差，形成河流网和湖泊⁴。地形稳定后，考虑雨影效应等因素再次调整降雨和温度（高山背风面降雨减少，森林区域降低温度波动），并进行最后的植被二次分配⁵。此时将确定各个连通的生物群落区域，给它们命名（如「阔叶林」、「高山草原」等），然后添加地质构造层（岩层和矿物分布）以及地下洞穴层，进行最后的参数验证⁶。生成通过后，系统为每个生物群落分配初始的野生动植物种群⁶。整个过程使用伪随机算法（DF中曾采用梅森旋转算法和SplitMix64等）⁷，并记录随机种子以便复现⁸⁶。

世界的生成结果包括多样的地形与生态：不同区域可能是高山、平原、森林、草原、沙漠、沼泽、海洋等各类生物群落，每种群落都有独特的气候、动植物与石层结构⁹。例如，山地生物群落石头层丰富、无土壤；温带阔叶林中丰富的树木和野生动物；沙漠区干燥贫瘠几乎无植被。每个群落还带有“善恶”属性及“狂野度”指标，决定该地的对立性和危险程度⁹¹⁰。世界生成时若不满足某些条件（如火山数量不足、生物群落分布不均等），会重新随机以“放弃并重试”的方式确保结果符合要求²¹¹。总的来说，矮人要塞以分形地形、流水侵蚀、气候模拟等多步骤的混合算法生成一个细致且一致的世界地图，并填充相应生态²⁴。

历史模拟系统

世界生成完成后，矮人要塞会启动历史模拟，将时间推进若干年，记录文明兴衰、战争迁徙、人物传记等事件。这个过程可视为一个**零玩家策略模拟**：游戏在背后让数千个代理（不同文明和野生生物）交互运行，模拟城镇建设、人口增长、战争征服等一系列动态过程¹²。开发者Tarn Adams本人将其形容为“一个零玩家战略游戏，松散的回合规则下有数千个代理，历史就是记录这一模拟产生的日志”¹²。在模拟过程中，各文明（矮人、人类、精灵、哥布林、蜥蜴人等）从各自的种族起源点扩张，它们会建立城镇、铸造要塞、采集资源、开辟道路。不同文明之间可能会因资源争夺或仇恨而爆发战争，城市被攻陷后可能被改名或沦为废墟¹²¹³。系统同时模拟人口生老病死、部落合并或分裂等社会演变，记录每位历史人物的出生、死亡、战功、诗歌或物品（如神器）等传记数据。玩家可以在传奇模式（Legends Mode）中查看完整历史档案，浏览各文明沿革、历史人物关系网和重大事件¹⁴¹²。

Tarn Adams在访谈中提到，早期历史生成并没有深入到每个动作的逻辑，而是让文明“到处繁衍、建城、扩散”，后期逐步增加了更多事件类型¹⁵¹⁶。例如，目前每个文明最初会从10对种族创始人开始（为避免近亲结婚问题）¹⁷，这些创始人及后续子孙将扩展建城，并逐渐形成各自的社会组织。随着模拟进行，系统会为每年发生的事件赋予文本描述：某苏丹征服城市、在战斗中获胜、与派系共建聚居点等等，这些事件构成了世界的背景故事¹³¹⁸。生成的历史既包括可见于游戏的地点（如废墟、古遗迹），也包括仅在文本中出现的背景信息（如某地区的帝国已灭亡）。总体而言，矮人要塞的历史模拟通过记录文明的动态演变和事件来丰富世界背景，让游戏世界具有“可浏览的传说记录”¹²¹⁹。

文明与社会系统

在矮人要塞中，各种族文明拥有丰富的社会文化要素和互动。每个文明内部有领导者（酋长、国王）、贵族等级和政府形态（君主制、部落制等），不同城市可能归属于不同的地方政权。人物层面，每个生物具有性别、年龄、技能、个性特征（如勇敢、悲观）等属性，并带有人际关系网络（朋友、敌人、家族关系）²⁰。个体记忆其经历过的事件，喜欢或厌恶特定物品，例如每个矮人可能有喜欢或讨厌的宝石类型²⁰。他们会形成宗教信仰：世界生成时还会产生众多神祇，每个神祇有光明或黑暗属性，居民可以崇拜多个神明，信仰深度分为热衷、虔诚、偶尔、冷淡等层级²¹²⁰。游戏中甚至会生成宗教派系（如敬虔的火焰教、可爱的信仰），并记录其起源和传承²²。

派系与阵营也是游戏重要部分：各文明、城市和氏族会形成不同的联盟或敌对关系，经常因为领土和资源发生冲突。神圣物品、史诗英雄和神器也会在历史中出现，并可能传承给某些家族或国家²³¹⁴。玩家在堡垒模式下还可以建立不同的社会结构，如议会或世袭制度，以及与邻近文明的外交（贸易、战争、联盟）。总体来看，矮人要塞的文明社会模拟覆盖了人物个体到整个文明层面的要素：人物的关系和情感、宗教文化、政治组织和派系冲突都被细致记录。

实现机制与代码设计

矮人要塞由独立开发者Tarn Adams用C和C++混合编写，迄今已超过70万行代码²⁴。它使用自研的游戏引擎，通过SDL和OpenGL实现跨平台图形输出²⁵，FMOD用于音效。游戏并未开源（源代码私有，由开发者维护），但支持二进制补丁和存档兼容。随机数生成方面，早年使用梅森旋转算法，近期改用SplitMix64以提高效率⁷。路径寻路使用A算法，但结合了连通域维护：游戏动态跟踪地图可通行区域，用连通分量标识，可快速判断路径可行性，避免对每次路径搜索均执行完整A²⁶。存档采用自定义的二进制格式，并需考虑不同操作系统字节序（开发者称存档格式从一开始就固定下来以应对大小端问题²⁷）。重要的数据结构方面，Tarn曾经尝试将物品体系改为类继承结构，但后来改用组合模式（类似组件系统）使物品更灵活；再后来也对地形、高度、运输等子系统进行了重构²⁸²⁶。总体上，矮人要塞的实现极度手工与粗犷，深度依赖巨量自定义逻辑和数据结构，由单人独立调试和维护²⁹。

《Caves of Qud》

世界生成机制

《Caves of Qud》的世界生成结合了固定预制元素和过程生成内容。游戏使用Unity引擎（C#编写）开发，核心世界名为“JoppaWorld”，其中包含已设计好的地块布局（如开场的Joppa村庄）和支持过程生成的区域³⁰³¹。大部分地区 and 地牢都是程序生成的：比如主线任务地点多数随机（如墓地、遗迹）、斯特利特（Stiltgrounds）周围的商店每次游玩都不同，³²；甚至诸如高尔哥萨（Golgotha）和食者之墓（Tomb of the Eaters）内部构造都包含大量随机生成元素³³。世界地图由很多“风息”（parasang）组成，每个风息相当于一个3×3屏幕的区域（3×3个屏幕称为一个风息），每个风息有一个表面生物群落类型（比如旷野、高地、沼泽、丛林等）³⁴。不同风息根据其生物群落类型决定表面出现的地形、植物和常见生物³⁵。表面区域还有深度分层：地下1到5层通常继承地表的生态特征，而更深层则仅与所在风息的难度等级（zone tier）相关³⁶。区域难度等级由风息的表面层级（zone tier）和当前位置深度共同决定³⁷。举例来说，“沙漠峡谷”地表必然包含特定的生物（如小蜥蜴、蜈蚣、蜻蜓等）；每个区域生成敌人或道具时，会根据该区域的难度等级抽取相应层级的列表³⁸³⁵。游戏还提供了可修改的XML配置（如Worlds.xml）定义世界蓝图和区域生成规则³⁰，可以通过编程方式在生成过程中插入新区域或迷踪点。总体而言，Qud的世界结构在地图布局上较为固定，但内容丰富且多样，结合了预制城镇和程序生成地牢，营造出多样化的超现实生态。

历史模拟系统

《Caves of Qud》并非在运行时持续模拟历史，而是在每个游戏开局时预先生成世界背景与传说。其核心机制是**苏丹传说（Sultan histories）**。游戏首先创作出五代（时期）的苏丹和他们的主要历史事件：每位苏丹会有随机生成的名字、人生大事和相关传说片段，并赋予“主题”标签（如某种生物或理念），贯穿其统治时代³⁹。固定的第六代苏丹Resheph始终存在，拥有不变的背景传说⁴⁰。传说生成以周期为单位：在第一代苏丹时期，系统随机生成2-3个新地区和6-8个新地点，并将它们纳入第一代苏丹的历史事件⁴¹¹³；第二代苏丹时期再生成2-3个地区和5-8个地点，以此类推，每代苏丹的可用地区地点不断累积⁴¹。在每个苏丹时代中，历史事件会涉及这些地区或地点：比如苏丹可能征服或解放一个城市，该地点会被以苏丹名义重新命名；或者苏丹联同某派系建新城镇¹³。这些事件不仅丰富了历史叙事，也直接决定游戏中地标的产生或改变。

在第四代和第五代苏丹阶段，还会各生成两个“历史遗迹”¹⁸。每个历史遗迹由苏丹支持的信徒（派系）居住守护，体现了苏丹偏爱的物种或文化。例如，某传说描述一位苏丹邂逅并学习了螃蟹部族的秘密，这暗示螃蟹族是其偏爱派系之一⁴²。这些生成的遗迹场景和其守护者直接与世界中的派系、宗教信仰相连。Qud的过程化历史还借鉴了口口相传和文献记录的叙事方式：传说内容允许视角偏差，每个记述都有可能加入夸张或偏见⁴³。总的来说，Qud通过在开局时生成多代苏丹与事件，创造了一个包含文明更替、战争胜利、城镇建立与毁灭、文化崇拜等元素的史诗背景¹³⁴³。

文明与社会系统

在Qud的世界里，各种派系（factions）和文化构成了社会网络。每个派系都有自己的信仰、生物种群和政治倾向，如螃蟹族（智慧甲壳生物）、沙漠僧侣、浸水人等。游戏开始生成历史时，每位苏丹都会与特定派系结盟或冲突，其偏爱的派系成员可能成为苏丹的臣民或追随者⁴²。这些派系在整个世界中散布，各自控制一定的据点或地盘。玩家角色也与派系互动：游戏跟踪角色在不同阵营中的声望（好感度），影响NPC的态度和任务选择⁴⁴。例如，玩家可能与某派系交好后获赠礼物或获得通行便利，与敌对派系则可能遭到伏击或价格暴涨。值得注意的是，Qud没有DF那样模拟每个普通NPC的详细生平，但其派系与苏丹历史赋予了世界深厚的社会背景和宗教氛围。例如，游戏中的“苏丹教派”遗迹及传说，实际上对应着玩家可以遇到的特定信仰团体。

综上，Qud的文明社会系统主要体现在**宏观派系关系**上：它通过程序生成的传奇和任务线将玩家与多样的阵营、信仰和文化联系在一起，而不是逐个模拟个体生活。游戏中的经济、政治元素多由固定或半固定的城镇生态体现（如城镇商人、定期展开的派系冲突），而世界史则通过文本和遗迹呈现种族和派系的碰撞与融合⁴²⁴⁴。

实现机制与代码设计

Caves of Qud采用Unity引擎（基于C#语言）开发⁴⁵，支持Windows、macOS和Linux平台⁴⁶。核心世界定义在XML文件中（如Worlds.xml），可通过编程API扩展世界生成过程³⁰。游戏并不完全开源，但对Mod制作支持充分：玩家可编写C#扩展类（如实现IJoppaWorldBuilderExtension接口）在地图生成前后插入自定义逻辑⁴⁷。随机性方面，Qud利用Unity或.NET提供的伪随机数生成器（带可重复种子）在世界生成时随机选取苏丹名字、事件、遗迹等，从而每次开始都产生不同的背景和地牢布局⁴³³⁹。

数据结构上，游戏将世界划分为“风息”单元，每个风息由多个地表和地下区块组成；生成时使用“区块构造器”（zone builder）在每个区块内创建地形、NPC和道具⁴⁸。物品、怪物等的生成有较完善的分层体系：区域的难度等级（zone tier）决定了生成表的档次³⁸。例如，高难度风息往往产出更强大的敌人和装备。游戏存档采用Unity默认方案，将存档文件保存在用户本地平台特定目录（如Windows系统%USERPROFILE%\AppData\LocalLow\Freehold Games\CavesOfQud\Synced\Saves⁴⁹），并通过Steam云同步实现多设备存档。配置文件和日志等也保存在类似路径⁴⁹。Caves of Qud并未公布完整源代码，但开放了内容

和事件的定义数据，使其在实现上比矮人要塞更现代：它通过Unity的组件系统和数据驱动方式构建世界，使用XML和脚本配置流程生成内容，并在游戏发布后持续更新和扩充生成算法^{50 51}。

对比分析

综上，两款游戏都强调程序生成世界与历史，但实现思路不同。矮人要塞主要追求**持续演化的模拟**：首先完全随机地生成地图和生态，然后模拟多年历史，让文明在动态系统中自行发展^{2 12}；它的³⁹世界是全新且自治的，每次生成都有独特而复杂的生态和文化背景。相对地，Caves of Qud的世界生成是**半固定半随机**：地图结构和城镇布局在多个游玩中保持大致相同，但通过每次游戏重新生成的苏丹故事和遗迹给世界赋予新内容³¹。

在历史模拟方面，DF采用零玩家模拟来产生事件日志¹²，而Qud则是将故事元素预生成并写入文本。DF的历史可以在传奇模式中动态继续，而Qud的“历史”主要服务于开局背景和任务生成。社会层面，DF模拟个人和群体（如人物的情感、家谱、宗教与政治）非常深入²⁰；Qud则更注重派系和故事，玩家与各派系的声誉互动是核心，而个人NPC并不具有同等复杂性^{44 42}。

从技术实现看，DF使用自研的C++引擎、专门的程序生成算法和复杂的数据结构，代码量巨大且闭源^{25 24}；Qud使用Unity+C#开发，依赖于引擎提供的框架与工具，生成逻辑则更多体现在配置和脚本中^{45 30}。DF的随机生成面向宏观世界演化，强调模拟真实感；Qud则通过程序叙事（procedural storytelling）融合了固定设定和随机元素，以创造叙事丰富的游戏体验^{43 39}。两者均采用伪随机技术，并允许玩家通过种子复现世界，但DF关注的是沙盒式“世界演化”，而Qud更像是以生成故事为目的的世界构建。

参考资料：有关矮人要塞的设计和实现细节可见开发者访谈与官方Wiki^{2 7 52}；关于《Caves of Qud》的世界和历史生成，可参考官方Wiki和开发者讲座^{31 39 53}。（所有引用格式均按照要求保留）

^{1 2 3 4 5 6 12 16 19} World generation - Dwarf Fortress Wiki

https://dwarffortresswiki.org/index.php/World_generation

^{7 20 23 24 25 26 27 28 29} 700,000 lines of code, 20 years, and one developer: How Dwarf Fortress is built - Stack Overflow

<https://stackoverflow.blog/2021/12/31/700000-lines-of-code-20-years-and-one-developer-how-dwarf-fortress-is-built/>

^{8 11} Advanced world generation - Dwarf Fortress Wiki

https://dwarffortresswiki.org/index.php/Advanced_world_generation

^{9 10} Biome - Dwarf Fortress Wiki

<https://dwarffortresswiki.org/index.php/Biome>

^{13 18 32 33 39 40 41 42} World generation - Official Caves of Qud Wiki

https://wiki.cavesofqud.com/wiki/World_generation

¹⁴ Legends - Dwarf Fortress Wiki

<http://dwarffortresswiki.org/legends>

^{15 17 52} Bay 12 Games: Dwarf Fortress Talk

https://www.bay12games.com/media/df_talk_4_transcript.html

²¹ DF2014:Religion - Dwarf Fortress Wiki

<https://dwarffortresswiki.org/index.php/DF2014:Religion>

²² Dev Log 13 Dec 2018: "There is always more to do, but we're at a ...

https://www.reddit.com/r/dwarffortress/comments/a638on/dev_log_13_dec_2018_there_is_always_more_to_do/

30 47 48 **Modding:Worlds - Official Caves of Qud Wiki**

<https://wiki.cavesofqud.com/wiki/Modding:Worlds>

31 43 44 45 46 51 **Caves of Qud - Wikipedia**

https://en.wikipedia.org/wiki/Caves_of_Qud

34 35 36 37 38 53 **Zone tier - Official Caves of Qud Wiki**

https://wiki.cavesofqud.com/wiki/Zone_tier

49 **File locations - Official Caves of Qud Wiki**

https://wiki.cavesofqud.com/wiki/File_locations

50 **Tapping into the potential of procedural generation in Caves of Qud**

<https://www.gamedeveloper.com/design/tapping-into-the-potential-of-procedural-generation-in-caves-of-qud>